



MACHINE
LEARNING
ACADEMY

MCP Fundamentals

ARCHITECTURE, SPECIFICATION & IMPLEMENTATION FOR DEVELOPERS

by

Peeya Iwagoshi

MCP Standardizes How AI Models Access Context

01 // THE CORE PROBLEM

Large Language Models (LLMs) operate in isolation, requiring developers to build custom integrations for every new data source or tool.

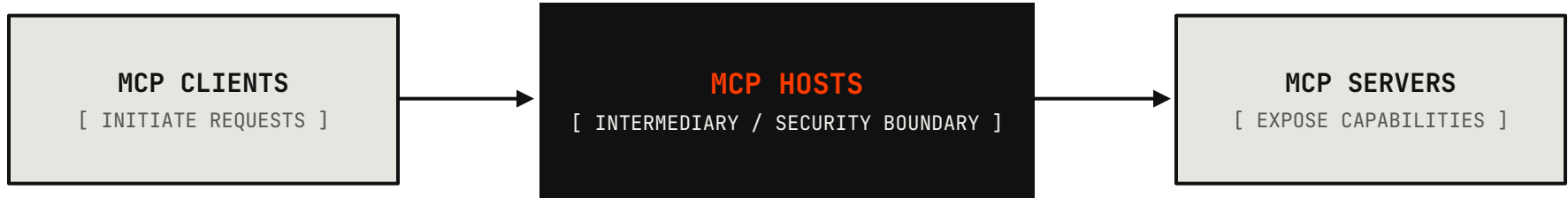
02 // THE MCP SOLUTION

An open standard introduced by Anthropic that provides a universal, standardized protocol for connecting AI assistants to external data sources.

03 // KEY BENEFIT

Developers write a single integration (an MCP Server) that instantly allows any compatible AI client to securely read data, execute tools, and retrieve dynamic prompts.

The Client-Host-Server Architecture Secure Routing



01 // CLIENTS

MCP Clients: The AI applications (like Claude Desktop or an IDE) that initiate requests and manage the user interface.

02 // HOSTS

MCP Hosts: The intermediary layer that routes requests, manages connections, and enforces security boundaries between the client and servers.

03 // SERVERS

MCP Servers: Lightweight, specialized programs that expose specific capabilities (e.g., a GitHub server, a PostgreSQL server, or a local file system server) to the Host.

04 // SECURITY

Separation of Concerns: This three-tier model ensures that AI models never directly touch backend data without explicit Host mediation.

Resources, Prompts, and Tools Form the Core

01 // RESOURCES

Data that the MCP Server exposes for the client to read. This includes files, database schemas, or API responses, typically identified by URI templates.

02 // PROMPTS

Pre-defined, reusable prompt templates managed by the server. They allow servers to dictate exactly how an AI should interact with specific domain data.

03 // TOOLS

Executable functions that the AI can invoke. The server defines the tool's schema (arguments and descriptions), and the host handles the actual execution authorization.

MCP Relies on JSON-RPC 2.0 and Flexible Transports

MESSAGE FRAMING

All MCP communications are formatted as standard JSON-RPC 2.0 messages, ensuring broad language compatibility and easy debugging.

STANDARD INPUT/OUTPUT (STDIO)

The most common transport for local integrations. The host spawns the server as a subprocess and communicates via stdin/stdout, ensuring zero network overhead.

SERVER-SENT EVENTS (SSE)

Used for remote or cloud-based MCP servers. HTTP POST requests handle client-to-server messages, while SSE handles the server-to-client stream.

SCHEMA VALIDATION

JSON Schema is heavily utilized to define tool arguments and resource metadata.

Connection Begins with Capabilities Negotiation

1

01 // INITIALIZATION

Initialization Phase: When a connection opens, the client and server exchange initialize requests to declare their supported protocol versions and capabilities.

Capability Mapping: The server explicitly lists if it supports resources, prompts, tools, or logging. The client acknowledges what it can handle.

2

02 // EXECUTION FLOW

Request/Response Flow: Once initialized, the client sends requests (e.g., tools/list or tools/call).

Processing: The server processes incoming execution requests synchronously or asynchronously based on workload.

3

03 // NOTIFICATIONS

One-Way Events: The protocol supports one-way notifications (e.g., notifications/resources/updated).

State Monitoring: Alerts clients of state changes in real-time without requiring active polling overhead.

Hosts Translate MCP into LLM Function Calls

1

SCHEMA DISCOVERY

The Host queries the MCP Server via `tools/list` to retrieve the JSON Schema for all available functions.

2

LLM INJECTION

The Host translates these schemas into the specific format required by the target LLM (e.g., Claude's tool use format or OpenAI's function calling).

3

EXECUTION ROUTING

When the LLM decides to use a tool, it outputs a tool call block. The Host intercepts this, formats it as an MCP `tools/call` request, and routes it to the correct server.

4

RESULT INJECTION

The server's response is passed back to the LLM as a new user message.

MCP Enforces Strict Client-Side Security and Auth

- **Local-First Execution:** By defaulting to **stdio** transports, MCP servers run locally with the permissions of the user, eliminating the need for complex cloud authentication.
- **Opt-In Tool Execution:** The Host application is responsible for asking the user for explicit permission before executing any state-changing tool (e.g., writing a file or dropping a table).
- **No Inbound Network Access:** Local MCP servers do not need to open network ports, drastically reducing the attack surface.
- **Granular Access:** Servers can be configured to restrict access to specific directories or database tables.

Practical Use Cases Span Enterprise and Local Workflows

01 // LOCAL DEVELOPER WORKFLOWS

Connecting IDEs to local file systems, Git repositories, and local development databases for context-aware coding assistance.

02 // ENTERPRISE SYSTEM INTEGRATION

Building MCP servers to safely query internal PostgreSQL databases, Salesforce CRM data, or internal wikis without exposing credentials to the LLM provider.

03 // SAAS INTEGRATIONS

Standardized servers for platforms like Slack, GitHub, and Google Drive, allowing AI assistants to read messages, review pull requests, and summarize documents seamlessly.

Developer SDKs MCP Server Implementation

// Official SDKs

Anthropic provides official SDKs for both **Python** and **TypeScript**, abstracting away the JSON-RPC framing and transport logic.

// Rapid Prototyping

Developers can build and test a functional MCP server connecting to a new API in **under 50 lines of code.**

mcp_server_implementation.ts

```
// Implementation Steps
// 1. Initialize Instance
const server = new MCPServer({...});

// 2. Define Tool Schemas (Zod / Pydantic)
const schema = z.object({...});

// 3. Register Handlers (tools/call, resources/read)
server.setRequestHandler(CallToolSchema, ...);

// 4. Start Server Transport
server.connect(new StdioServerTransport());
```

The MCP Ecosystem Is Driving Towards Open Standardization

01 // GROWING OPEN-SOURCE REGISTRY

The community is rapidly building open-source MCP servers for common tools (SQLite, Puppeteer, GitHub, Brave Search).

02 // CLIENT ADOPTION

Major AI clients, including Claude Desktop, Cursor, and Zed, are actively integrating MCP Host capabilities.

03 // FUTURE OUTLOOK

MCP aims to become the universal standard for AI context, much like USB standardized hardware peripherals or LSP (Language Server Protocol) standardized IDE tooling.

04 // INTEROPERABILITY

The ultimate goal is seamless interoperability across any LLM provider and any client application.

Key Takeaways & Next Steps

STANDARDIZED CONTEXT

MCP eliminates custom integration code by providing a unified protocol for Resources, Prompts, and Tools.

SECURE BY DESIGN

Local-first architecture and Host-mediated execution keep user data safe.

1. Review the Official Specification

Read details at modelcontextprotocol.io

2. Install the SDK

Install the Python or TypeScript @mcp/sdk

3. Build Your First MCP Server

Expose a custom API or database to your AI assistant.

4. Explore Existing Integrations

Check the open-source MCP server registry for tools.

Local MCP Server Using STDIO for Communication

```
# server.py - using STDIO
import sys
from mcp.server.fastmcp import FastMCP

# Initialize FastMCP server
mcp = FastMCP("Simple Calculator")

# MCP decorator
@mcp.tool()
def add_numbers(a: float, b: float) -> str:
    """Adds two numbers together and returns the
    result."""
    # Log to stderr to avoid breaking the JSON-RPC
    # stdout channel
    print(f"Executing add_numbers with a={a}, b={b}",
          file=sys.stderr)
    return str(a + b)

if __name__ == "__main__":
    # Run the server using default STDIO transport
    mcp.run()
```

```
# client.py - Client side
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def call_stdio_server():
    server_params = StdioServerParameters(
        command="python",
        args=["server.py"], env=None,
        # You can inject API keys or PATH variables here
    )

    async with stdio_client(server_params) as (read_stream, write_stream):
        async with ClientSession(read_stream, write_stream) as session:

            await session.initialize()

            print("Sending tool execution call...")
            result = await session.call_tool(
                name="add_numbers",
                arguments={"a": 12.5, "b": 7.5} )

            for content_item in result.content:
                if content_item.type == "text":
                    print(f"Server Response: {content_item.text}")

if __name__ == "__main__":
    asyncio.run(call_stdio_server())
```

MCP Communication Initialization

```
# Initialization request from  
# (MCP client to MCP server)
```

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "method": "initialize",  
  "params": {  
    "protocolVersion": "2025-11-05",  
    "capabilities": {},  
    "clientInfo": {  
      "name": "raw-client",  
      "version": "1.0.0"  
    }  
  }  
}
```

```
# Request for the list of tools (MCP Client -> Server)
```

```
{  
  "jsonrpc": "2.0",  
  "id": 1  
  "method": "tools/list",  
}
```

```
# Response of the tool list request (MCP Server -> Client)
```

```
{  
  "jsonrpc": "2.0",  
  "id": 1  
  "result": {  
    "tools": [  
      {  
        "name": "add_numbers",  
        "description": "Adds two numbers together and returns the result.",  
        "inputSchema": {  
          "type": "object",  
          "properties": {  
            "a": { "type": "number" },  
            "b": { "type": "number" }  
          },  
          "required": ["a", "b"]  
        }  
      }  
    ]  
  },  
}
```

MCP Agent to Tool Communication

jsonrpc: Always "2.0". This specifies the protocol standard being used.

method: The protocol action being executed (e.g., `tools/list`, `tools/call`, `resources/list`, or `prompts/list`).

params: Contains the execution payload. Inside it, `name` targets the specific Python function, and `arguments` contains the key-value pairs matching your function parameters.

id: A unique string or integer managed by the client. The server *must* echo this exact `id` back in its response so the client knows which specific request is being answered (especially vital if messages are processed asynchronously).

content: An array of outputs returned to the LLM. MCP natively supports multiple types of content inside this array, such as `"text"` or `"image"`.

```
# Invoke the tools (MCP Client -> Server)
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "add_numbers",
    "arguments": {
      "a": 10.5,
      "b": 4.5
    }
  },
  "id": 2
}
```

```
# Response from the tool (MCP Server -> Client)
{
  "jsonrpc": "2.0",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "15.0"
      }
    ],
    "isError": false
  },
  "id": 2
}
```

MCP Agent to Model Communication

In a standard agent loop, the MCP Client reads information from your MCP server (schemas, resource text, or tool outputs) and forwards it to the LLM's standard provider API (like the Anthropic, OpenAI, or Google Gemini API). [\[1\]](#), [\[2\]](#)

When the MCP client structures this payload for the LLM, the "call" contains:

- **System Prompt Customization:** Instructions detailing how the LLM should behave and how it should respect the injected data. [\[1\]](#)
- **The Available Tools Array:** A JSON translation of your server's tool schemas (derived from your python code), detailing the name, descriptive intent, and parameter object shapes the model can utilize. [\[1\]](#), [\[2\]](#)
- **The Chat Messages History:** The rolling conversation, containing standard text payloads along with `tool_use` and `tool_result` message blocks mapping back to the data your Docker container generated. [\[1\]](#)

MCP Agent to Model Communication

messages: An array containing the immediate conversation history or structured prompt data requiring LLM completion. [1]

systemPrompt: A distinct system instruction scoped strictly to this internal micro-task. [1]

modelPreferences: Strategic metadata allowing the server to tell the client what kind of model capability it needs (e.g., targeting a high *intelligencePriority* vs. a low *costPriority*). [1]

includeContext: Controls whether the client should inject additional localized context (*none*, *thisServer*, or *allServers*) into the eventual prompt before hitting the model endpoint. [1]

⚠ Protocol Update Note: The *sampling/createMessage* method was officially deprecated in the Model Context Protocol (MCP) specification. Because MCP transitioned to a stateless architecture, servers can no longer push ad-hoc requests back to clients over a standard connection. [1, 2, 3]

```
# Invoke the model (MCP Client -> Model Server)
{
  "jsonrpc": "2.0",
  "method": "sampling/createMessage",
  "params": {
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "Analyze the following raw server logs
and extract the error root cause..."
        }
      }
    ],
    "systemPrompt": "You are an expert infrastructure
reliability engineer.",
    "modelPreferences": {
      "hints": [{ "name": "gemini-3.5-flash" }],
      "costPriority": 0.3,
      "speedPriority": 0.5,
      "intelligencePriority": 0.9
    },
    "includeContext": "thisServer",
    "temperature": 0.2,
    "maxTokens": 1024
  },
  "id": "sampling-req-001"
}
```

MCP Network Communication

LAYER 2: The HTTPS Wrapper (Application Layer) - HTTP POST request sent to the server's dedicated MCP endpoint (e.g., `https://api.example.com/mcp`).

LAYER 3: The TLS Session (Transport Security Layer) - Because it runs over HTTPS, the entire plain-text HTTP block above is completely encrypted before hitting TCP. It is broken down into **TLS Record Layer packets**.

HTTP request

```
POST /mcp HTTP/1.1
Host: api.example.com
User-Agent: <Agent Name>
Content-Type: application/json; charset=utf-8
Accept: application/json, text/event-stream
Authorization: Bearer mcp_live_token_abcdef123456
Content-Length: 298
Connection: keep-alive
```

```
{"jsonrpc": "2.0", "method": "tools/call", "params": {"name": "add_numbers", "arguments": {"a": 10.5, "b": 4.5}, "_meta": {"protocolVersion": "2026-07-28"}}, "id": "req-98745"}
```

TLS Record Header (5 bytes)	
- Content Type: 23 (Application Data)	
- Version: 0x0303 (TLS 1.2) or 0x0304 (TLS 1.3)	
- Length: Variable (e.g., 512 bytes)	
+-----+	
Encrypted Payload (AES-GCM or CHACHA20-POLY1305)	
[Includes HTTP Headers + MCP JSON text block]	
+-----+	
Authentication Tag (MAC / MAC Checksum)	
+-----+	
TCP(Layer 4) / IP (Layer 5) / Ethernet Frame (Layer 6)	
+-----+	



References

[Model Context Protocol Documentation](#)

[The MCP Protocol: Transports, Primitives, and the Message Lifecycle](#)

[MCP Examples in Github](#)

Architecture of MCP Examples

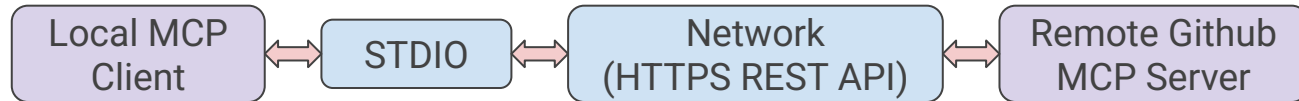
- **Examples 1 and 2:**



- **Examples 3 (git):**



- **Example 4 (Github):**



- **Example 5 (Model call):** The client to model connection is the conduit for the model to call the MCP Server.

